Report: Rediscovering Heartbleed (CVE-2014-0160) with libFuzzer

Samat Iakupov, Dmitrii Antipov, Mark Petrov

Where the screenshots come from: the log file `demo/fuzz-demo.log` (full output of one `make docker-fuzz` run with `tee`; this file has **1475 lines**). For each figure below we give **line numbers in that file**. Turn on line numbers in your editor and copy the range we give. If you record a new log, line numbers may move—find the same text using the figure captions.

---

1. Abstract

This is a course-style **fuzzing** project. The goal was to **find again a known bug**: **CVE-2014-0160 (Heartbleed)** in **OpenSSL 1.0.1f**. We used **libFuzzer** and **AddressSanitizer (ASan)** inside **Docker** (Ubuntu 22.04, clang). Random bytes go into a TLS **server** context through in-memory BIOs. When the parser hits the bad heartbeat path, ASan reports a **heap-buffer-overflow** with a stack frame in `tls1_process_heartbeat` (`ssl/t1_lib.c`). libFuzzer saves the input as a **crash file**; you can **replay** it by running the fuzzer binary on that file.

---

2. Task and choices

Task: pick a fuzzing tool from the allowed list (libFuzzer, AFL++, Google FuzzTest), run it, and show **rediscovery** of a known CVE on a **vulnerable** build.

Why this CVE: CVE-2014-0160 is famous, well documented, and there are public examples (including ClusterFuzz).

Why this tool: libFuzzer + ASan gives fast in-process fuzzing, one instrumented binary, and works well with clang in the container.

---

3. How we built the project (step by step)

1. **Read about the CVE.** Heartbleed is a bad TLS Heartbeat handler: the code trusts the **length** field in the message without checking that the message really contains that many bytes. Then it **reads past the end** of the buffer.

2. **Pick a target.** We pinned **OpenSSL 1.0.1f** (still vulnerable; fixed in 1.0.1g).

3. **Repo layout.** We added:

   - `Dockerfile` — base image, packages, build OpenSSL + fuzz target during `docker build`;
   - `Makefile` — targets `build`, `fuzz-build`, `fuzz`, `reproduce`, `docker-fuzz`, `docker-reproduce`;
   - `scripts/build_openssl.sh` — download source tarball, build static `libssl.a` / `libcrypto.a` with ASan and `fuzzer-no-link`;

- `fuzz/heartbleed_fuzz.cc` — harness with `LLVMFuzzerTestOneInput`, TLS server, `SSL_do_handshake` / `SSL_read`;
- `scripts/run_fuzz.sh` — run libFuzzer with corpus + dictionary;
- `scripts/generate_corpus.py` — write starter seed files;
- `fuzz/heartbleed.dict` — byte tokens for TLS / heartbeat.

4. **First Docker run.** We used `make docker-fuzz` to build the image and start fuzzing.

5. **Fix a link error.** On some setups, linking `heartbleed_fuzz` with static `libssl.a` and `libcrypto.a` failed with many **`undefined reference`** errors to symbols that live in `libcrypto`. Fix: wrap both archives in **`-Wl,--start-group`** … **`-Wl,--end-group`** in the `Makefile` so the linker can resolve symbols across the static libs.

6. **Small script hardening (quality).**

   - `build_openssl.sh`: fail clearly if download fails; check the archive is non-empty; check **SHA256** of the tarball (override with `OPENSSL_TARBALL_SHA256` if needed); use `sha256sum` or `shasum` depending on the OS;
   - `reproduce.sh`: if no crash path is passed, pick the **newest** `crash-*` file by time, not the first name in alphabetical order.

7. **Final run.** After the image built, we ran `make docker-fuzz` and saved the output with `tee demo/fuzz-demo.log` for the report and demo.

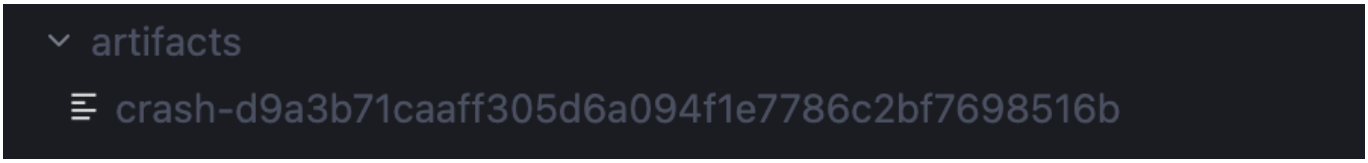
8. **Replay check.** `make docker-reproduce` replays a crash from `artifacts/` and should show the same ASan report. You can save that output to e.g. `demo/reproduce-demo.log` and cite line ranges the same way.

---

4. Repository layout (short)

Path	Role
`Dockerfile`	Image with toolchain; prebuilds OpenSSL + fuzz target
`Makefile`	Build and run (local or Docker)
`fuzz/heartbleed_fuzz.cc`	Fuzz harness (libFuzzer entry)
`fuzz/corpus/`	Starter corpus (created at build time)
`fuzz/heartbleed.dict`	libFuzzer dictionary
`scripts/build_openssl.sh`	Download and build OpenSSL 1.0.1f
`scripts/run_fuzz.sh`	libFuzzer flags (`-dict`, `-max_total_time`, `artifact_prefix`)
`scripts/reproduce.sh`	Replay one saved crash file
`artifacts/`	On the host: mounted volume with `crash-*` from the container
`demo/fuzz-demo.log`	Full run log for the report and figures

Optional figure (not from the log). Screenshot of `ls -la artifacts/` on the host so the `crash-<sha>` file is visible.



---

5. Building OpenSSL and the fuzzer

OpenSSL is built as **static** libs (`no-shared`) with:

- `-fsanitize=address,fuzzer-no-link` — instrument the libs for ASan and for linking with the fuzzer, without linking the fuzzer runtime into the library itself.

Fuzzer is a separate `clang++` binary with `-fsanitize=address,fuzzer` and static `libssl.a` / `libcrypto.a` inside a **link group** (see below).

Snippet from the `Makefile` (main part):

```
clang++ -g -O1 -std=c++17 \
  -fsanitize=address,fuzzer \
  -I$(OPENSSL_DIR)/include \
  fuzz/heartbleed_fuzz.cc \
  -Wl,--start-group \
  $(OPENSSL_DIR)/libssl.a \
  $(OPENSSL_DIR)/libcrypto.a \
  -Wl,--end-group \
  -ldl -lpthread -o $(FUZZER)
```

To check “build works and we hunt the bug”:

```
make docker-fuzz
```

The command should end with a **non-zero exit code**: when ASan finds a bug it **aborts** the process. That is **not** a failed build; it means the sanitizer fired.

Figure 1. Part of the log from the `docker build` step: corpus generation, the `clang++` link line for `heartbleed_fuzz` with `-Wl,--start-group` / `--end-group`, copy of certs, end of layer `#10 DONE`.

- **File:** `demo/fuzz-demo.log`
- **Lines: 1337–1351** (from `mkdir -p build` / `python3 scripts/generate_corpus.py` through `#10 DONE 57.1s`)

```
1337  #10 55.73 make[1]: Leaving directory '/work/openssl-1.0.1f/crypto'
1338  #10 55.74 mkdir -p build fuzz/crashes
1339  #10 55.88 python3 scripts/generate_corpus.py
1340  #10 55.95 clang++ -g -O1 -std=c++17 \
1341  #10 55.95   -fsanitize=address,fuzzer \
1342  #10 55.95   -Iopenssl-1.0.1f/include \
1343  #10 55.95   fuzz/heartbleed_fuzz.cc \
1344  #10 55.95   -Wl,--start-group \
1345  #10 55.95   openssl-1.0.1f/libssl.a \
1346  #10 55.95   openssl-1.0.1f/libcrypto.a \
1347  #10 55.95   -Wl,--end-group \
1348  #10 55.95   -ldl -lpthread -o build/heartbleed_fuzz
1349  #10 56.96 cp fuzz/server.pem build/server.pem
1350  #10 56.96 cp fuzz/server.key build/server.key
1351  #10 DONE 57.1s
```

Figure 2. Docker exports the image and tags `heartbleed-fuzz:latest` — proof the image built before `docker run`.

- **File:** `demo/fuzz-demo.log`
- **Lines: 1353–1366** (from `#11 exporting to image` through the `View build details:` line)

```
1353  #11 exporting to image
1354  #11 exporting layers
1355  #11 exporting layers 3.4s done
1356  #11 exporting manifest sha256:aba1b36be5c91ec3f69bc12ffd0353b668e39d9df6a81fa704a084c4622e946b
1357  #11 exporting manifest sha256:aba1b36be5c91ec3f69bc12ffd0353b668e39d9df6a81fa704a084c4622e946b done
1358  #11 exporting config sha256:03f2d37082e4cd181e332b7d857f123ca36bbd43732ac0a568f007ad4ca5b8ae done
1359  #11 exporting attestation manifest sha256:70b469039fba6d79d312730a60cd52dde990973a3091f49a5e3a8e4f59f6f27a 0.0s done
1360  #11 exporting manifest list sha256:2d55545aa39678ce135c0f405ae92a532a15fe82e25749f821e03ec0ca5db8e2 0.0s done
1361  #11 naming to docker.io/library/heartbleed-fuzz:latest done
1362  #11 unpacking to docker.io/library/heartbleed-fuzz:latest
1363  #11 unpacking to docker.io/library/heartbleed-fuzz:latest 1.1s done
1364  #11 DONE 4.5s
1365
1366  View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/bn3uf41xluydxx0vy2my20m55
```

---

6. Harness (what we actually fuzz)

Harness (`fuzz/heartbleed_fuzz.cc`):

1. `LLVMFuzzerInitialize` stores the directory of the executable (to load `server.pem` / `server.key` next to the binary in `build/`).
2. Once: create server `SSL_CTX` for TLSv1 and load the test certificate.
3. In **`LLVMFuzzerTestOneInput`** for each input:
   - create `SSL*`, in/out **memory BIOs**;
   - write fuzz bytes into the inbound BIO;
   - call **`SSL_do_handshake`**, then loop **`SSL_read`** to drain records.

So OpenSSL treats the bytes as TLS server traffic; with the right mutation, control reaches the heartbeat handler and the **1.0.1f** bug triggers.

Figure 3. Stack snippet at crash time: the path goes from `LLVMFuzzerTestOneInput` (`heartbleed_fuzz.cc:101`) through `SSL_do_handshake` into OpenSSL—so the harness really drives the TLS stack.

- **File:** `demo/fuzz-demo.log`
- **Lines: 1392–1400** (from `#0 __asan_memcpy` through `#7 ... LLVMFuzzerTestOneInput ... heartbleed_fuzz.cc:101`)

```
READ of size 32064 at 0xffffb6009748 thread T0
    #0 0xaaaabd274b5c in __asan_memcpy (/work/build/heartbleed_fuzz+0x2a4b5c) (BuildId: 3eaca696e6dd42bf5241d2c2cec22
    #1 0xaaaabd2bc740 in tls1_process_heartbeat /work/openssl-1.0.1f/ssl/t1_lib.c:2586:3
    #2 0xaaaabd31fbdc in ssl3_read_bytes /work/openssl-1.0.1f/ssl/s3_pkt.c:1092:4
    #3 0xaaaabd3251f8 in ssl3_get_message /work/openssl-1.0.1f/ssl/s3_both.c:457:7
    #4 0xaaaabd2f50c8 in ssl3_get_client_hello /work/openssl-1.0.1f/ssl/s3_srvr.c:941:4
    #5 0xaaaabd2f1e08 in ssl3_accept /work/openssl-1.0.1f/ssl/s3_srvr.c:357:9
    #6 0xaaaabd2d063c in SSL_do_handshake /work/openssl-1.0.1f/ssl/ssl_lib.c:2564:7
    #7 0xaaaabd2af9dc in LLVMFuzzerTestOneInput /work/fuzz/heartbleed_fuzz.cc:101:3
    #8 0xaaaabd1e6604 in fuzzer::Fuzzer::ExecuteCallback(unsigned char const*, unsigned long) (/work/build/heartbleed
```

---

7. Corpus and dictionary

- **Seeds:** a small heartbeat-like record and a larger ClientHello-like blob (`scripts/generate_corpus.py`) — they help the fuzzer reach “TLS-shaped” data faster.
- **Dictionary** (`heartbleed.dict`): heartbeat type `0x18`, TLS version bytes, handshake prefixes — helps mutations hit the right format sooner.

Figure 4. Fuzzer startup: `docker run`, `./scripts/run_fuzz.sh`, `Dictionary:`, corpus info (`2 files`, seed sizes), libFuzzer `INFO` lines and a few mutation lines right **before** the ASan banner (you can see dictionary use, e.g. `ManualDict` / `heartbeat_request`).

- **File:** `demo/fuzz-demo.log`
- **Lines: 1367–1388** (from `mkdir -p ".../artifacts"` / `docker run` through the `#50 REDUCE` line, the last line before the `=================================================================` banner)

*Note: In the log file this block appears **before** the stack in Figure 3, because it is earlier in the same run.*

```
mkdir -p "/Users/samerspc/Documents/enterprise_js/SSD-project/ssd_project/artifacts"
docker run --rm  -v "/Users/samerspc/Documents/enterprise_js/SSD-project/ssd_project/artifacts:/artifacts" -e FUZZ_CRA
./scripts/run_fuzz.sh
Dictionary: 7 entries
INFO: Running with entropic power schedule (0xFF, 100).
INFO: Seed: 659121209
INFO: Loaded 1 modules   (36165 inline 8-bit counters): 36165 [0xaaaabd745eb0, 0xaaaabd74ebf5),
INFO: Loaded 1 PC tables (36165 PCs): 36165 [0xaaaabd74ebf8,0xaaaabd7dc048),
INFO:        2 files found in /work/fuzz/corpus
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 4096 bytes
INFO: seed corpus: files: 2 min: 8b max: 221b total: 229b rss: 49Mb
#3  INITED cov: 515 ft: 593 corp: 2/229b exec/s: 0 rss: 50Mb
  NEW_FUNC[1/1]: 0xaaaabd3b0584 in ERR_put_error /work/openssl-1.0.1f/crypto/err/err.c:708
#4  NEW    cov: 521 ft: 632 corp: 3/245b lim: 221 exec/s: 0 rss: 50Mb L: 16/221 MS: 1 InsertRepeatedBytes-
#5  NEW    cov: 521 ft: 639 corp: 4/261b lim: 221 exec/s: 5 rss: 50Mb L: 16/221 MS: 1 ChangeByte-
#8  NEW    cov: 522 ft: 640 corp: 5/271b lim: 221 exec/s: 8 rss: 50Mb L: 10/221 MS: 3 InsertByte-ChangeBinInt-EraseByt
#14 NEW    cov: 522 ft: 643 corp: 6/276b lim: 221 exec/s: 14 rss: 51Mb L: 5/221 MS: 1 EraseBytes-
#30 NEW    cov: 522 ft: 649 corp: 7/383b lim: 221 exec/s: 30 rss: 51Mb L: 107/221 MS: 1 InsertRepeatedBytes-
#41 NEW    cov: 525 ft: 655 corp: 8/604b lim: 221 exec/s: 41 rss: 52Mb L: 221/221 MS: 1 ManualDict- DE: "heartbeat_req
#42 REDUCE cov: 525 ft: 655 corp: 8/598b lim: 221 exec/s: 42 rss: 52Mb L: 10/221 MS: 1 EraseBytes-
#48 REDUCE cov: 525 ft: 655 corp: 8/506b lim: 221 exec/s: 48 rss: 52Mb L: 129/221 MS: 1 CrossOver-
#50 REDUCE cov: 525 ft: 683 corp: 9/516b lim: 221 exec/s: 50 rss: 52Mb L: 10/221 MS: 2 ShuffleBytes-ChangeBit-
```

---

8. Fuzzing run — what we saw

Command: `make docker-fuzz` (inside: `docker run` with `artifacts/` mounted, then `make fuzz` → `scripts/run_fuzz.sh`).

On a good run:

- libFuzzer loads corpus + dictionary (`Dictionary: … entries`);
- coverage grows (`NEW`, `REDUCE`, `cov` / `ft`);
- ASan prints **`heap-buffer-overflow`**;
- stack shows **`tls1_process_heartbeat`** and **`ssl/t1_lib.c:2586`**;
- libFuzzer writes **`Test unit written to /artifacts/crash-<sha>`**;
- `make` fails with **`ABORTING`** — that is **expected**.

Figure 5. Main proof: ASan header, error type, stack with **`tls1_process_heartbeat`** and **`t1_lib.c:2586`**.

- **File:** `demo/fuzz-demo.log`
- **Lines: 1389–1408** (from the `====` line and `ERROR: AddressSanitizer` through frame `#16 _start`)

```
1389  =================================================================
1390  ==7==ERROR: AddressSanitizer: heap-buffer-overflow on address 0xffffb6009748 at pc 0xaaaabd274b60 bp 0xffffb014370 sp
1391  READ of size 32064 at 0xffffb6009748 thread T0
1392      #0 0xaaaabd274b5c in __asan_memcpy (/work/build/heartbleed_fuzz+0x2a4b5c) (BuildId: 3eaca696e6dd42bf5241d2c2cec22e
1393      #1 0xaaaabd2bc740 in tls1_process_heartbeat /work/openssl-1.0.1f/ssl/t1_lib.c:2586:3
1394      #2 0xaaaabd31fbdc in ssl3_read_bytes /work/openssl-1.0.1f/ssl/s3_pkt.c:1092:4
1395      #3 0xaaaabd3251f8 in ssl3_get_message /work/openssl-1.0.1f/ssl/s3_both.c:457:7
1396      #4 0xaaaabd2f50c8 in ssl3_get_client_hello /work/openssl-1.0.1f/ssl/s3_srvr.c:941:4
1397      #5 0xaaaabd2f1e08 in ssl3_accept /work/openssl-1.0.1f/ssl/s3_srvr.c:357:9
1398      #6 0xaaaabd2d063c in SSL_do_handshake /work/openssl-1.0.1f/ssl/ssl_lib.c:2564:7
1399      #7 0xaaaabd2af9dc in LLVMFuzzerTestOneInput /work/fuzz/heartbleed_fuzz.cc:101:3
1400      #8 0xaaaabd1e6604 in fuzzer::Fuzzer::ExecuteCallback(unsigned char const*, unsigned long) (/work/build/heartbleed_
1401      #9 0xaaaabd1e5e18 in fuzzer::Fuzzer::RunOne(unsigned char const*, unsigned long, bool, fuzzer::InputInfo*, bool, b
1402      #10 0xaaaabd1e74f4 in fuzzer::Fuzzer::MutateAndTestOne() (/work/build/heartbleed_fuzz+0x2174f4) (BuildId: 3eaca696
1403      #11 0xaaaabd1e8154 in fuzzer::Fuzzer::Loop(std::vector<fuzzer::SizedFile, std::allocator<fuzzer::SizedFile> >&) (/
1404      #12 0xaaaabd1d71b4 in fuzzer::FuzzerDriver(int*, char***, int (*)(unsigned char const*, unsigned long)) (/work/bui
1405      #13 0xaaaabd1fe6ac in main (/work/build/heartbleed_fuzz+0x22e6ac) (BuildId: 3eaca696e6dd42bf5241d2c2cec22e2f6ec7a5
1406      #14 0xffffbd9773fc  (/lib/aarch64-linux-gnu/libc.so.6+0x273fc) (BuildId: 9f32d43c341bff10b9e7196738eedcfc4f3cc36c)
1407      #15 0xffffbd9774d4 in __libc_start_main (/lib/aarch64-linux-gnu/libc.so.6+0x274d4) (BuildId: 9f32d43c341bff10b9e71
1408      #16 0xaaaabd1cd5ec in _start (/work/build/heartbleed_fuzz+0x1fd5ec) (BuildId: 3eaca696e6dd42bf5241d2c2cec22e2f6ec7
```

Figure 6. Memory context: heap region text, **allocation** stack (`CRYPTO_malloc`, `ssl3_setup_read_buffer`, …), and the **`SUMMARY: AddressSanitizer`** line.

- **File:** `demo/fuzz-demo.log`
- **Lines: 1410–1430** (from `0xffff... is located` through `SUMMARY: AddressSanitizer: heap-buffer-overflow ...`)

```
0xffffb6009748 is located 0 bytes to the right of 17736-byte region [0xffffb6005200,0xffffb6009748)
allocated by thread T0 here:
    #0 0xaaaabd275788 in malloc (/work/build/heartbleed_fuzz+0x2a5788) (BuildId: 3eaca696e6dd42bf5241d2c2cec22e2f6ec7a
    #1 0xaaaabd34f140 in default_malloc_ex /work/openssl-1.0.1f/crypto/mem.c:79:11
    #2 0xaaaabd34ff30 in CRYPTO_malloc /work/openssl-1.0.1f/crypto/mem.c:308:8
    #3 0xaaaabd3269fc in freelist_extract /work/openssl-1.0.1f/ssl/s3_both.c:708:12
    #4 0xaaaabd3269fc in ssl3_setup_read_buffer /work/openssl-1.0.1f/ssl/s3_both.c:770:10
    #5 0xaaaabd326f90 in ssl3_setup_buffers /work/openssl-1.0.1f/ssl/s3_both.c:827:7
    #6 0xaaaabd2f27f0 in ssl3_accept /work/openssl-1.0.1f/ssl/s3_srvr.c:292:9
    #7 0xaaaabd2d063c in SSL_do_handshake /work/openssl-1.0.1f/ssl/ssl_lib.c:2564:7
    #8 0xaaaabd2af9dc in LLVMFuzzerTestOneInput /work/fuzz/heartbleed_fuzz.cc:101:3
    #9 0xaaaabd1e6604 in fuzzer::Fuzzer::ExecuteCallback(unsigned char const*, unsigned long) (/work/build/heartbleed_
    #10 0xaaaabd1e7938 in fuzzer::Fuzzer::ReadAndExecuteSeedCorpora(std::vector<fuzzer::SizedFile, std::allocator<fuzz
    #11 0xaaaabd1e7efc in fuzzer::Fuzzer::Loop(std::vector<fuzzer::SizedFile, std::allocator<fuzzer::SizedFile> >&) (/
    #12 0xaaaabd1d71b4 in fuzzer::FuzzerDriver(int*, char***, int (*)(unsigned char const*, unsigned long)) (/work/bui
    #13 0xaaaabd1fe6ac in main (/work/build/heartbleed_fuzz+0x22e6ac) (BuildId: 3eaca696e6dd42bf5241d2c2cec22e2f6ec7a5
    #14 0xffffbd9773fc  (/lib/aarch64-linux-gnu/libc.so.6+0x273fc) (BuildId: 9f32d43c341bff10b9e7196738eedcfc4f3cc36c)
    #15 0xffffbd9774d4 in __libc_start_main (/lib/aarch64-linux-gnu/libc.so.6+0x274d4) (BuildId: 9f32d43c341bff10b9e71
    #16 0xaaaabd1cd5ec in _start (/work/build/heartbleed_fuzz+0x1fd5ec) (BuildId: 3eaca696e6dd42bf5241d2c2cec22e2f6ec7

SUMMARY: AddressSanitizer: heap-buffer-overflow (/work/build/heartbleed_fuzz+0x2a4b5c) (BuildId: 3eaca696e6dd42bf5241d
Shadow bytes around the buggy address:
```

Figure 7. End of run: `ABORTING`, hex dump of the triggering input, **crash file path**, libFuzzer stats, `make: *** [fuzz] Error 1`. In the caption, state clearly: **a non-zero `make` exit here means success** (the process was stopped on purpose by ASan).

- **File:** `demo/fuzz-demo.log`
- **Lines: 1459–1474** (from `==7==ABORTING` through `make: *** [docker-fuzz] Error 2`)

```
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==7==ABORTING
MS: 1 InsertByte-; base unit: 9ba56b277afc43462bcc8fd399680c9048a9683f
0x18,0x3,0x2,0x0,0x3,0x1,0x7d,0x40,0x0,
\030\003\002\000\003\001}@\000
artifact_prefix='/artifacts/'; Test unit written to /artifacts/crash-d9a3b71caaff305d6a094f1e7786c2bf7698516b
Base64: GAMCAAMBfUAA
stat::number_of_executed_units: 51
stat::average_exec_per_sec:     51
stat::new_units_added:          9
stat::slowest_unit_time_sec:    0
stat::peak_rss_mb:              53
make: *** [Makefile:41: fuzz] Error 1
make: *** [docker-fuzz] Error 2
```

Replay:

```
make docker-reproduce
```

This runs the fuzzer on the saved file under `artifacts/`; you should see the same error class and the same code path.

Figure 8 (optional). Screenshot from a separate log, e.g. `make docker-reproduce 2>&1 | tee demo/reproduce-demo.log` — again `AddressSanitizer` and `tls1_process_heartbeat`. Line numbers depend on output length; crop from `Running:` / `ERROR: AddressSanitizer` through `ABORTING`.

---

9. Mapping the crash to CVE-2014-0160

- **Version:** OpenSSL 1.0.1f is in the vulnerable range before 1.0.1g.
- **Mechanism:** out-of-bounds **read** while handling heartbeat (Figures **5–6** show `heap-buffer-overflow` / read in `__asan_memcpy` from `tls1_process_heartbeat`).
- **Code location:** `tls1_process_heartbeat` is the documented Heartbleed site.
- **Replay:** the same input crashes again (Figure **7** — saved `crash-*`; Figure **8** — optional replay log).

That is the assignment's "rediscover a known CVE with fuzzing," not a random unrelated crash.

---

10. Patch (reference only)

Upstream fix: commit `96db9023b881d7cd9f379b0c154650d6c108e9a3` — adds bounds checks before copying the claimed heartbeat payload; bad messages are dropped per RFC 6520.

---

11. Conclusions

- A small **harness** around a real library lets libFuzzer quickly hit "wrong trust in length fields" style protocol bugs.
- **ASan** turns an out-of-bounds **read** into a loud failure with a stack—without it, Heartbleed would stay a quiet memory leak.
- **Docker** pins OS and compiler and makes grading / reproduction easier.
- Static OpenSSL needed careful **archive order / link group** settings for the fuzz target.

---

12. Quick table: figure → section → lines in `demo/fuzz-demo.log`

Figure	Report section	Lines in `demo/fuzz-demo.log`
Fig. 1	§5 Build	**1337–1351**
Fig. 2	§5 Build	**1353–1366**
Fig. 3	§6 Harness	**1392–1400**
Fig. 4	§7 Corpus & dictionary	**1367–1388**
Fig. 5	§8 Results	**1389–1408**
Fig. 6	§8 Results	**1410–1430**
Fig. 7	§8 Results	**1459–1474**
Fig. 8	§8 (optional)	separate file `reproduce-demo.log`

---

13. References

1. CVE-2014-0160 (MITRE/NVD): https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160
2. OpenSSL security advisory (2014-04-07): https://www.openssl.org/news/secadv/20140407.txt
3. Fix commit: https://github.com/openssl/openssl/commit/96db9023b881d7cd9f379b0c154650d6c108e9a3

4. ClusterFuzz Heartbleed example: https://google.github.io/clusterfuzz/setting-up-fuzzing/heartbleed-example/
5. OpenSSL 1.0.1 source archive: https://www.openssl.org/source/old/1.0.1/